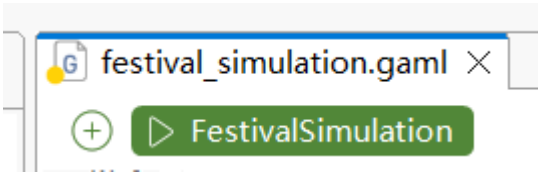
ID2209 Assignment 2: Auction

Group 3
Shunkang Jia, Rui Zhang, Yuning Liu
shunkang, rzha, yuningl @kth.se

Section 1: Running Instruction

Copy the*.gaml file into user model section and click the “FestivalSimulation” button.
Each task has its own gaml file, overall 3 files are submitted.



Section 2: General Overview

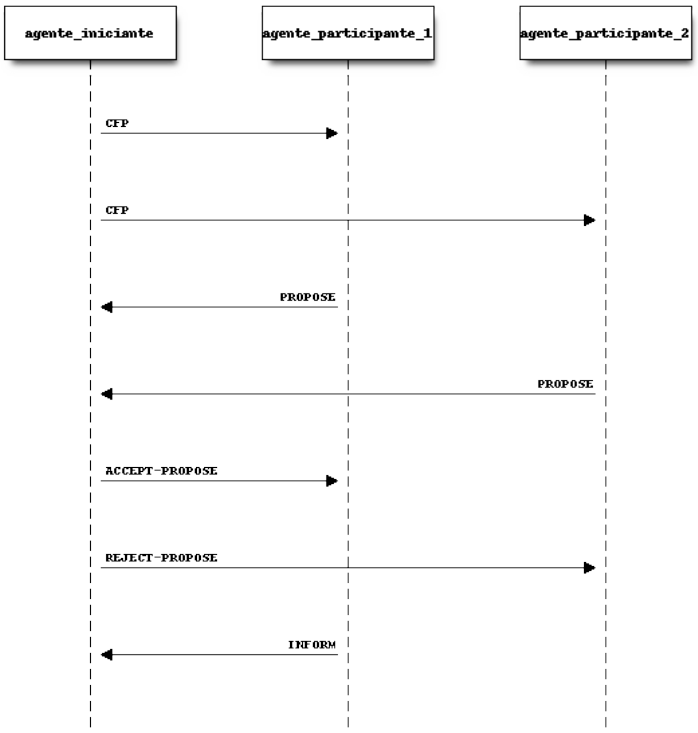
In this assignment we have finished all mandatory parts and also challenge 1 and 2.

We first implement a basic dutch auction with only 1 genre and 1 auctioneer with no-protocol.

Iteratively, we introduced more genres and allowed parallel dutch auctions on different items and also enabled guests to join the auction that they are interested in.

Finally, we developed an English auction and a sealed-bit auction and compare the buyer price.

Following is an illustration of the fipa net contract protocol.



Section 3: Auction System

3.1 Explanation

This section required introducing a new interaction mechanism inside the festival simulation:

a Dutch auction where a dedicated agent communicates with Guests using the FIPA communication protocol.

Our approach:

I implemented a full Dutch auction cycle:

· The Auctioneer starts from a high price and sends CFP messages to all Guests.

· Guests compare the current auction price with their personal max_price.

· If the price is acceptable, they respond using FIPA propose.

· The Auctioneer selects the first proposal received, confirms it with accept_proposal, and stops the auction.

3.2 Code

3.2.1 Creating the Auctioneer

A single Auctioneer is created near the information center and is responsible for managing the auction.

```
        //创建拍卖师
        create Auctioneer number:1{
            location <- info_center_location + {10,0};
        }
    }
```

3.2.2 Guest – Auction Attributes & Response Logic

A Guest sends a FIPA propose message only if the price is acceptable and the Guest has not already participated in the auction.

```
 // ===== 拍卖: Guest 回应 Auctioneer 的 cfp=====
reflex respond_to_auction when: !empty(cfps) {

    // 只要收到了 CFP, 就看看当前拍卖价格
    Auctioneer a <- one_of(Auctioneer);         // 这里只有一个拍卖师

    float price_from_auction <- a.last_announced_price;

    // 只对自己感兴趣、且价格 <= max_price 时出价
    if (auction_genre = a.genre
        and price_from_auction <= max_price
        and !in_auction) {

        do propose
            message: cfps[length(cfps) - 1]     // 用最新那条 CFP 做
            contents: ["buy", price_from_auction];

        in_auction <- true;
        write name + " proposes to buy " + a.genre
              + " at price " + price_from_auction;
    }
}
```

3.2.3 Auctioneer – Dutch Auction & Winner Selection

The Auctioneer runs the Dutch auction by lowering the price at fixed intervals and broadcasting each new price through FIPA CFP messages. Guests independently evaluate the announced price and may respond with a FIPA propose message when it falls within their acceptable range. Once a proposal is received, the Auctioneer confirms the winner using accept_proposal and ends the auction.

```
// ===== 拍卖师代理=====
species Auctioneer skills: [fipa] {

    string genre <- "CD";           // 拍卖物品类别（先写死，之后 Challenge 
    float start_price <- 120.0;     // 起拍价（高于市场价）
    float min_price <- 40.0;        // 最低价
    float price_step <- 10.0;       // 每轮降价幅度

    float current_price <- start_price;
    float last_announced_price <- start_price;

    bool auction_running <- true;  // simulation 一开始就开始拍卖
    int last_cfp_time <- 0;        // 上一次发 cfp 的时间

    Guest winner <- nil;
    float final_price <- 0.0;

    aspect default {
        draw circle(1.5) color: #yellow border: #black;
        draw "A" at: {location.x, location.y + 2} color: #black;
    }

    // 每隔 5 步发一次 cfp，价格往下掉（荷兰拍）
    reflex run_dutch_auction when: auction_running and (time - last_cfp

        if (current_price < min_price) {
            write "Auctioneer: auction cancelled, price < min_price.";
            auction_running <- false;
        } else {
            write "Auctioneer: sending CFP at price " + current_price;
            last_announced_price <- current_price;

            // 发 cfp 给所有 Guest（使用 FIPA 的 start_conversation）
            do start_conversation (
                to: list(Guest),
                protocol: "no-protocol",
                performative: "cfp",
                contents: [genre, current_price]
            );
```

```
                contents: [genre, current_price]
            );

            current_price <- current_price - price_step;
            ⊗ last_cfp_time of type int is assigned a value of type flo
            last_cfp_time <- time;
        }
    }

    // 收到 Guest 的 propose（有人愿意买）
    reflex handle_proposals when: auction_running and !empty(proposes)

        Guest g <- proposes[0].sender as Guest;
        winner <- g;
        final_price <- last_announced_price;

        write "Auctioneer: winner is " + g.name
            + " at price " + final_price;

        // 用 FIPA 的 accept_proposal 回复
        do accept_proposal (
            message: proposes[0],
            contents: ["accepted", final_price]
        );

        auction_running <- false;
    }
}
```

3.3 Demonstration

3.3.1 Use case 1 —- Successful Dutch Auction

(1) Input Description

In this scenario, the Auctioneer starts the Dutch auction at a price of 120 and decreases it by 10 each round. All Guests have a randomly assigned maximum

acceptable price between 60 and 120. The auction continues until the announced price becomes acceptable to at least one Guest.

(2) Screenshot

```
Information Center: Bad guest reported!
Security Guard: Bad guest removed from festival!
Auctioneer: sending CFP at price 120.0
Auctioneer: sending CFP at price 110.0
Guest0 proposes to buy CD at price 110.0
Guest4 proposes to buy CD at price 110.0
Guest7 proposes to buy CD at price 110.0
Guest13 proposes to buy CD at price 110.0
Auctioneer: winner is Guest0 at price 110.0
Information Center: Bad guest reported!
Information Center: Bad guest reported!
Security Guard: Bad guest removed from festival!
Security Guard: Bad guest removed from festival!
Information Center: Bad guest reported!
Security Guard: Bad guest removed from festival!
```

(3) Interpretation

This output shows a successful Dutch auction. The initial price of 120 is too high, so no Guest proposes. When the price decreases to 110, several Guests find the offer acceptable and send FIPA propose messages. The Auctioneer follows the Dutch auction rule of selecting the *first proposer*—Guest0—as the winner and confirms the sale through an accept_proposal message. This verifies that the CFP broadcasting, Guest bidding behavior, and winner selection mechanisms all function correctly.

3.3.2 Use case 2 —- Auction Cancelled

(1) Input Description

In this scenario, all Guests are assigned a very low maximum acceptable price, such that none of them are willing to purchase the auctioned item. The Auctioneer starts at a price of 120 and decreases the price by 10 each round. Because all Guests' max_price values are below the entire price range, no Guest ever sends a FIPA propose message.

(2) Screenshot

```
Auctioneer: sending CFP at price 120.0
Auctioneer: sending CFP at price 110.0
Auctioneer: sending CFP at price 100.0
Auctioneer: sending CFP at price 90.0
Auctioneer: sending CFP at price 80.0
Auctioneer: sending CFP at price 70.0
Auctioneer: sending CFP at price 60.0
Auctioneer: sending CFP at price 50.0
Auctioneer: sending CFP at price 40.0
Auctioneer: auction cancelled, price < min_price.
```

(3) Interpretation

This output illustrates the situation where no Guest accepts the auction price at any stage. The Auctioneer continuously broadcasts CFP messages while lowering the price from 120 down to 40. Because all Guests have a maximum acceptable price far below the announced prices, no propose messages are received. When the next price would drop below the minimum threshold (40), the Auctioneer correctly terminates the auction by printing “auction cancelled”. This confirms that the system handles the “no buyer” case safely and completes the auction without errors.

Section 4: Challenge 1: Multiple Auctions in the Festival

4.1 Explanation

Here is our auction setting: The Dutch Auction is applied. There are a total of 2 auctioneers and 6 guests (bidder) (Numbers can be changed) . Each guest has several preferred genres and expected values for each genre. Each guest can only join 1 auction at a time.

There are four types of items. Each has a specific pricing strategy defined by a **Starting Price** (High), a **Minimum Price** (Low), and a **Decrement Step** (how much the price drops per round).

Item Type	**Color**	**Starting Price**	**Decrement Step**	**Minimum Price**	**Expect Value(Guest)**
T-Shirt	Red	800.0	150.0	200.0	200.0 – 800.0
Cap	Yellow	400.0	30.0	100.0	100.0 – 400.0
CD	Blue	50.0	5.0	10.0	10.0 – 50.0
Poster	Green	20.0	2.0	3.0	3.0 – 20.0

4.2 Code

4.2.1 The FIPA Protocol Flow

The interaction follows the `fipa-contract-net` pattern, modified for a Dutch Auction:

1. **[Auctioneer] Call for Proposal (CFP):** The Auctioneer broadcasts an invitation containing the `auction_id` and `item_genre`.

```
// Step 1: Invite participants
reflex start_auction when: !auction_active and flip(0.1) {
    auction_active <- true;
    auction_id <- name + "_" + cycle;
    item_genre <- one_of(item_genres);

    // 不同商品不同定价策略
    switch item_genre {
        match "CD" { starting_price <- 50.0; price_decrement <- 5.0; minimum_price <- 10.0; }
        match "Poster" { starting_price <- 20.0; price_decrement <- 2.0; minimum_price <- 3.0; }
        match "Cap" { starting_price <- 400.0; price_decrement <- 30.0; minimum_price <- 100.0; }
        match "T-Shirt" { starting_price <- 800.0; price_decrement <- 150.0; minimum_price <- 200.0; }
    }

    current_price <- starting_price;
    participants <- [];
    winner <- nil;
    timer <- 0;
    first_round <- true;

    write "--------------------------------------------------";
    write "--- NEW AUCTION: " + item_genre + " ---";
    write "--- Strategy: Start Price " + starting_price + " | Drop Rate " + price_decrement + " | Min " + minimum_price + " ---";

    do start_conversation to: list(guest) protocol: 'fipa-contract-net' performative: 'cfp'
        contents: ['auction_id', auction_id, 'genre', item_genre];
```

2. **[Guest] Propose (Join):** Interested Guests reply with an `interested` status to join the "room".

```
// 1. Receive Invitation (Join the Room)
reflex receive_cfp when: !empty(cfps) {
    loop cfpMsg over: cfps {
        list msg_content <- cfpMsg.contents;
        string auction_id <- string(msg_content[1]);
        string item_genre <- string(msg_content[3]);

        // Decision: Do I like this genre? (Price is unknown yet)
        if ((item_genre in valuations.keys) and !in_auction) {
            write name + ": Joining room for " + item_genre;
            do propose message: cfpMsg contents: ['interested', true, 'auction_id', auction_id];

            // 更新状态 (这里删除了 color <- #green，改为状态控制)
            in_auction <- true;
            current_auction_id <- auction_id;
            current_item_type <- item_genre;
        } else {
            do refuse message: cfpMsg contents: ['interested', false];
        }
    }
}
```

3. **[Auctioneer] Propose (Price Broadcast):** The Auctioneer broadcasts the current price to joined participants.

```
// Step 2: Conduct the Auction Loop
reflex conduct_auction when: auction_active and !empty(participants) {
    timer <- timer + 1;

    if (timer >= price_update_interval) {

        // 2a. Check if anyone bought
        if (!empty(accept_proposals)) {
            message winning_msg <- first(accept_proposals);
            winner <- guest(winning_msg.sender);

            write ">>> SOLD: " + item_genre + " to " + winner.name + " for " + current_price;

            do inform message: winning_msg contents: ['result', 'won', 'final_price', current_price, 'auction_id', auction_id];

            list<guest> losers <- participants - winner;
            loop loser over: losers {
                do start_conversation to: [loser] protocol: 'fipa-contract-net' performative: 'inform' contents: ['result', 'lost', 'auction_id', auction_id];
            }
            do end_auction;
            return;
        }

        // 2b. Prepare next price
        if (first_round) {
            write name + ": First call -> " + current_price;
            first_round <- false;
        } else {
            current_price <- current_price - price_decrement;
            write name + ": Dropping Price -> " + current_price;
        }

        // 2c. Check Minimum Price
        if (current_price < minimum_price) {
            write "--- CANCELLED: Reserve not met (Current: " + current_price + " < Min: " + minimum_price + ") ---";
            loop participant over: participants {
                do start_conversation to: [participant] protocol: 'fipa-contract-net' performative: 'inform' contents: ['result', 'cancelled', 'auction_id', auction_id];
            }
            do end_auction;
        } else {
            // 2d. Broadcast Price
            loop participant over: participants {
                do start_conversation to: [participant] protocol: 'fipa-contract-net' performative: 'propose'
                    contents: ['price', current_price, 'auction_id', auction_id];
            }
        }

        timer <- 0;
    }
}
```

4. **[Guest] Accept/Reject:**
    - Guests evaluate the price. If acceptable, they send an `accept_proposal`.
    - If too expensive, they send `reject_proposal` (or simply wait for the next lower price).

```
// 2. Receive Price Proposal (The actual bidding)
reflex receive_propose when: !empty(proposes) {
    loop proposeMsg over: proposes {
        list msg_content <- proposeMsg.contents;
        float current_price <- float(msg_content[1]);
        string auction_id <- string(msg_content[3]);

        if (auction_id != current_auction_id) { continue; }

        float my_valuation <- valuations[current_item_type];

        // Logic: Buy if Price <= Valuation AND Price <= Budget
        if (current_price <= my_valuation and current_price <= budget) {
            write name + ": !!! BUYING at " + current_price + " (Val: " + my_valuation + ") !!!";
            do accept_proposal message: proposeMsg contents: ['accept', true, 'bid', current_price];
        } else {
            // Dutch Auction silence implies rejection usually, but we act polite
            do reject_proposal message: proposeMsg contents: ['accept', false];
        }
    }
}
```

5. **[Auctioneer] Inform:** The Auctioneer ends the auction by informing the winner (`won`) and the losers (`lost`). (Code is in auction loop - step 3)

4.2.2 Agent Strategies

Guest Strategy

- **Initialization:** During setup, guests are assigned random valuations for items. They have a 70% chance of being interested in any given category.

```
create guest number: nb_guests {
    location <- {rnd(100), rnd(100)};

    // 随机生成该 Guest 对不同商品的兴趣和心理价位
    loop genre over: item_genres {
        // 70% 的几率对某商品感兴趣
        if (flip(0.7)) {
            float min_val <- 0.0;
            float max_val <- 0.0;
            if (genre = "Poster") { min_val <- 3.0; max_val <- 20.0; }
            else if (genre = "CD") { min_val <- 10.0; max_val <- 50.0; }
            else if (genre = "Cap") { min_val <- 100.0; max_val <- 400.0; }
            else if (genre = "T-Shirt") { min_val <- 200.0; max_val <- 800.0; }
            else { min_val <- 10.0; max_val <- 100.0; }

            valuations[genre] <- rnd(min_val, max_val);
        }
    }
```

- **Decision to Join:** When receiving a CFP, the guest checks `valuations.keys`. If the item is in their interest list and they are not currently in another auction, they join.
- **Bidding Logic:** inside the `receive_propose` reflex:

  IF (Current_Price <= My_Valuation) AND (Current_Price <= My_Budget)
  THEN -> Send ACCEPT (Buy immediately)
  ELSE -> Send REJECT (Wait for price drop)

Auctioneer Strategy

- **Winner Selection:** The auctioneer checks the `accept_proposals` list.
  - Since the Dutch auction ends on the **first** bid, the auctioneer selects the first message in the list as the winner (`first(accept_proposals)`).
  - The item is sold at the `current_price`.
- **Price Update:** If no one accepts:
  - `current_price` is reduced by `price_decrement`.
  - If `current_price < minimum_price`, the auction is cancelled, and all participants are notified.

4.3 Demonstration

4.3.1 Initial Logs

Every time we start a simulation, we print the guests budget, interested genres and the maximum price they are willing to pay for a genre.

```
--------------------------------------------------
--- SYSTEM INITIALIZATION: GUEST PROFILE ANALYSIS ---
--------------------------------------------------
GUEST [guest0] Total Budget: 1702
    Item: T-Shirt -> Not Interested
    Item: CD -> Interested (Psychological Val: 46)
    Item: Poster -> Not Interested
    Item: Cap -> Interested (Psychological Val: 229)

GUEST [guest1] Total Budget: 712
    Item: T-Shirt -> Interested (Psychological Val: 778)
    Item: CD -> Interested (Psychological Val: 48)
    Item: Poster -> Interested (Psychological Val: 12)
    Item: Cap -> Interested (Psychological Val: 399)

GUEST [guest2] Total Budget: 824
    Item: T-Shirt -> Interested (Psychological Val: 633)
    Item: CD -> Interested (Psychological Val: 31)
    Item: Poster -> Interested (Psychological Val: 18)
    Item: Cap -> Interested (Psychological Val: 345)
```

4.3.2 Successful Sale

A normal dutch auction have the following output logs.

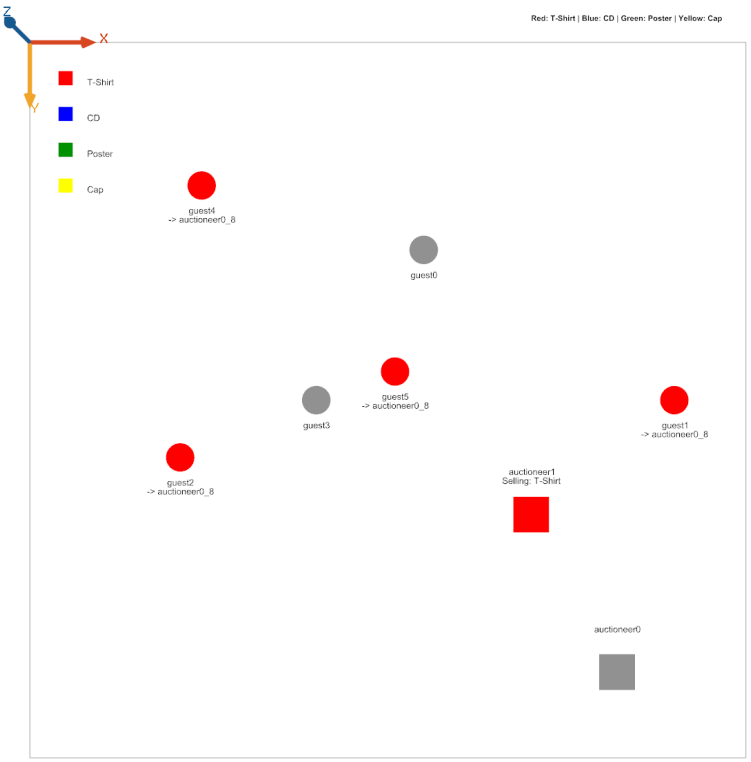
```
--------------------------------------------------
--- NEW AUCTION: Cap ---
--- Strategy: Start Price 400.0 | Drop Rate 30.0 | Min 100.0 ---
guest0: Joining room for Cap
guest1: Joining room for Cap
guest2: Joining room for Cap
guest3: Joining room for Cap
guest4: Joining room for Cap
guest5: Joining room for Cap
auctioneer0: First call -> 400.0
auctioneer0: Dropping Price -> 370.0
guest1: !!! BUYING at 370.0 (Val: 399.67304179941243) !!!
guest3: !!! BUYING at 370.0 (Val: 376.80930184740345) !!!
>>> SOLD: Cap to guest1 for 370.0
--------------------------------------------------
```

4.3.3 Uninterested Customers

In this challenge, guests only attend the auction for genres that they are interested in. We make a clear visualization and output log for it. To illustrate, when an auctioneer starts an auction. It changes color from grey (not activated) into the color of that genre. Then guests that are interested also change from grey to that color, representing that they are joining the auction.

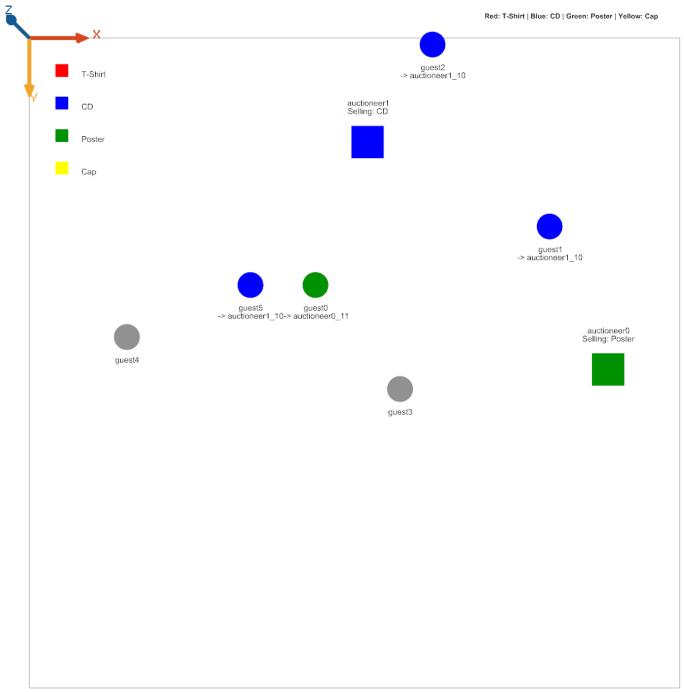
As is shown in this scenario, only guest 1,2,4,5 are interested in T-shirt, represented by red

```
--- NEW AUCTION: T-Shirt ---
--- Strategy: Start Price 800.0 |
guest1: Joining room for T-Shirt
guest2: Joining room for T-Shirt
guest4: Joining room for T-Shirt
guest5: Joining room for T-Shirt
```



4.3.4 Parallel Auctions

In our implementation, sometimes there may be 2 auctions in parallel. In this case auctioneer 0 and 1 happened to start the auction at the same time and guests can only choose one to attend.



Section 5 ：Challenge 2: Different Auction Settings

5.1 Explanation

In this challenge, here is our auction setting: The Dutch Auction, English Auction and Sealed Bid Auction are applied. There are a total of 6 auctioneers and 20 guests (bidder) (Numbers can be changed) .

5.2 Code

5.2.1 Guest

```
// ==================== Guest ====================
species Guest skills: [moving, fipa] {
    float hunger <- 100.0 min: 0.0 max: 100.0;
    float thirst <- 100.0 min: 0.0 max: 100.0;
    // 降低阈值，让他们更愿意参加拍卖而不是去吃饭
    float hunger_threshold <- 20.0;
    float thirst_threshold <- 20.0;

    string state <- "idle" among: ["idle", "seeking_info", "going_to_store", "at_store", "in_auction"];
    point target_location <- nil;
    Store target_store <- nil;
    bool is_bad <- false;
    bool reported <- false;
    bool has_memory <- true;
    list<Store> memory <- [];
    point last_location <- location;

    // 拍卖相关
    float money <- 500.0;
    list<string> interests <- [];
    agent current_auction <- nil;
    float auction_market_value <- 0.0;  // 保存当前拍卖的市场价值
```

5.2.2 Auction logic

```
// ==================== 拍卖逻辑 (使用 Helper 解析) ====================

// 1. 收到 CFP (邀请)
reflex listen_to_cfps when: state != "in_auction" and !empty(cfps) {
    loop msg over: cfps {
        list c <- list(msg.contents);
        string genre <- get_string_from_content(c, "genre");
        string auc_type <- get_string_from_content(c, "auction_type");
        float market_val <- get_float_from_content(c, "market_value");

        // 如果感兴趣且有钱 (放宽条件)
        if ((interests contains genre) and money > 50.0) {
            current_auction <- msg.sender;
            auction_market_value <- market_val;  // 保存市场价值
            state <- "in_auction"; // 锁定状态

            do start_conversation with: [
                to: [msg.sender], protocol: "fipa-contract-net", performative: "propose",
                contents: ["interested", true]
            ];
            write name + " 参加 " + auc_type + " (" + genre + ")";
            // 参加一个就够了，跳出循环
            break;
        }
    }
}

// 2. 收到 Propose (Dutch报价 或 SealedBid请求 或 English出价结果)
reflex handle_proposes when: state = "in_auction" and !empty(proposes) {
    loop msg over: proposes {
        if (msg.sender = current_auction) {
            list c <- list(msg.contents);

            // Dutch Logic
            float d_price <- get_float_from_content(c, "current_price");
            if (d_price > 0) {
                if (d_price <= money * 0.9) {
                    do start_conversation with: [to: [current_auction], protocol: "fipa-contract-net", performative: "accept_proposal", co
                    write name + " 接受 Dutch 价格: " + d_price;
                }
                return;
            }

            // English Logic (Response to my bid usually, but here implies outbid or new round)
            // English is handled mostly via INFORMS in this model

            // Sealed Logic (Request to submit)
            string action_req <- get_string_from_content(c, "action");
            if (action_req = "submit_bid") {
                float market_val <- get_float_from_content(c, "market_value");
                float my_bid <- rnd(market_val * 0.5, min(money, market_val * 1.2));
                do start_conversation with: [to: [current_auction], protocol: "fipa-contract-net", performative: "propose", contents: ["se
                write name + " 提交密标: " + my_bid;
            }
        }
```

```
// 3. 收到 Inform (English 更新 或 结果通知)
reflex handle_informs when: state = "in_auction" and !empty(informs) {
    loop msg over: informs {
        if (msg.sender = current_auction) {
            list c <- list(msg.contents);

            // 检查是不是拍卖结束/取消
            string status <- get_string_from_content(c, "status");
            agent winner <- agent(c[1]); // 假设winner在第二个位置，或者我们用解析

            // 如果消息包含 "winner" 键 (我们在 Auctioneer 里是这么发的)
            // 这里简化处理：如果收到 status=ended 或 cancelled，重置状态
            if (status = "ended" or status = "cancelled") {
                state <- "idle";
                current_auction <- nil;

                // 检查自己是否赢了
                loop i from: 0 to: length(c) - 1 step: 2 {
                    if (string(c[i]) = "winner" and agent(c[i+1]) = self) {
                        float price <- get_float_from_content(c, "final_price");
                        string a_type <- get_string_from_content(c, "auction_type");
                        money <- money - price;

                        // 正确计算买家盈余 = 市场价值 - 实际支付
                        float buyer_gain <- auction_market_value - price;
                        ask world { do register_buyer_value(a_type, buyer_gain); }

                        write name + " 赢得拍卖! 花费: " + price + " (市场价: " + auction_market_value + ", 盈余: " + buyer_gain + ")";

                        // 重置市场价值
                        auction_market_value <- 0.0;
                    }
                }
                return;
            }

            // English Auction Update
            float highest <- get_float_from_content(c, "current_highest_bid");
            float inc <- get_float_from_content(c, "min_increment");

            if (highest > 0) {
                float next_bid <- highest + inc;
                if (next_bid < money and flip(0.6)) {
                    do start_conversation with: [to: [current_auction], protocol: "fipa-contract-net", performative: "propose", contents: ["bid", next_bid]];
                    write name + " English 加价到: " + next_bid;
                }
            }
        }
    }
}

// 4. 收到 Reject (English 出价被拒/无效)
reflex handle_rejects when: state = "in_auction" and !empty(reject_proposals) {
    // 仅仅是这次出价失败，不需要退出拍卖状态
}

aspect default {
    draw circle(1.5) color: (state="in_auction") ? #gold : (state="idle" ? #gray : #blue);
```

5.2.3 Auctioneer

```
// ==================== Auctioneer ====================
species Auctioneer skills: [fipa] {
    string auction_type;
    string item_genre;
    float item_market_value;
    float starting_price;
    float current_price;
    float min_price;
    float price_decrement;
    float min_increment;

    int start_cycle_offset; // 随机偏移量

    string state <- "preparing";
    int auction_round <- 0;
    list<agent> participants <- [];
    agent winner <- nil;
    float final_price <- 0.0;

    float highest_bid <- 0.0;
    agent highest_bidder <- nil;
    int no_bid_rounds <- 0;
    map<agent, float> sealed_bids <- [];

    // 状态机逻辑
    reflex fsm {
        // 1. 准备阶段：每 30 个周期尝试一次，或者刚开始时 (cycle = offset)
        if (state = "preparing") {
            if (cycle = start_cycle_offset or (cycle > start_cycle_offset and (cycle - start_cycle_offset) mod 30 = 0)) {
                state <- "announcing";
                participants <- [];
                winner <- nil;
                final_price <- 0.0;
                highest_bid <- 0.0;
                highest_bidder <- nil;
                sealed_bids <- [];

                // 重置价格
                current_price <- starting_price;
                if (auction_type = "English") { highest_bid <- starting_price; }

                write ">>> [NEW AUCTION] " + name + " starts " + auction_type + " for " + item_genre;

                do start_conversation with: [
                    to: list(Guest), protocol: "fipa-contract-net", performative: "cfp",
                    contents: ["auction_type", auction_type, "genre", item_genre, "starting_price", starting_price, "market_value", item_marke
                ];
            }
        }

        // 2. 收集参与者阶段
        else if (state = "announcing") {
            loop msg over: proposes {
                if !(participants contains msg.sender) { add msg.sender to: participants; }
            }

            // 等待2个周期收集回复
```

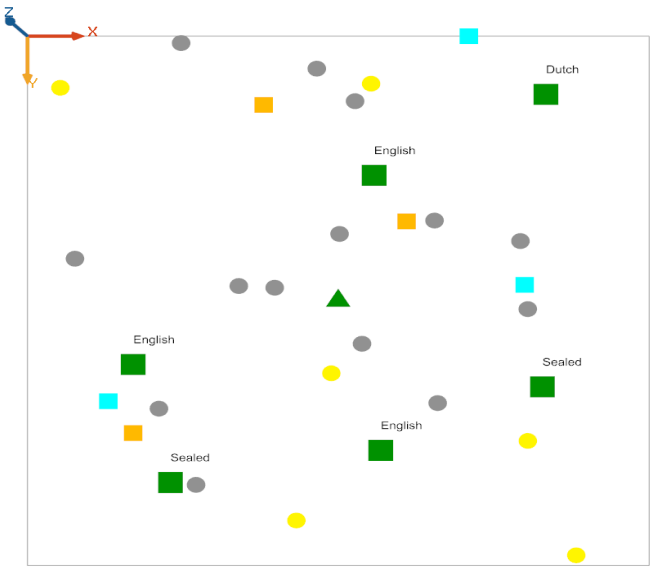
```
                    highest_bid <- bid;
                    highest_bidder <- msg.sender;
                    new_bid <- true;
                }
            }
        }
    }

    if (!new_bid) {
        no_bid_rounds <- no_bid_rounds + 1;
        if (no_bid_rounds >= 3 and highest_bidder != nil) {
            winner <- highest_bidder;
            final_price <- highest_bid;
            do end_auction;
        } else if (no_bid_rounds >= 3) {
            do cancel_auction;
        }
    } else {
        no_bid_rounds <- 0;
    }
}

action run_sealed {
    loop msg over: proposes {
        list c <- list(msg.contents);
        // 简易解析
        loop i from: 0 to: length(c)-1 step: 2 {
            if (string(c[i]) = "sealed_bid") {
                sealed_bids[msg.sender] <- float(c[i+1]);
            }
        }
    }

    if (length(sealed_bids) > 0 and auction_round > 3) {
        // 找最高
        float max_p <- 0.0;
        agent best_a <- nil;
        loop a over: sealed_bids.keys {
            if (sealed_bids[a] > max_p) { max_p <- sealed_bids[a]; best_a <- a; }
        }
        winner <- best_a;
        final_price <- max_p;
        do end_auction;
    }
}

action end_auction {
    state <- "preparing"; // 回到准备状态，等待下一轮
    auction_round <- 0;

    // 全局统计
    ask world { do register_revenue(myself.auction_type, myself.final_price); }

    // 通知所有人
    do start_conversation with: [
        to: participants, protocol: "fipa-contract-net", performative: "inform",
        contents: ["status", "ended", "winner", winner, "final_price", final_price, "auction_type", auction_type]
    ];
```

5.3 Demonstration

5.3.1 Logs

```
Guest7 English bids up to: 95.17356307058142
Guest17 English bids up to: 95.17356307058142
>>> [Global Stats] Auction Success! Type: Sealed Amount: 218.92852
Auctioneer5 Ended. Winner: Guest(9) Price: 218.92852452030095
Guest3 English bids up to: 95.17356307058142
Guest7 English bids up to: 95.17356307058142
Guest9 won auction! Cost: 218.92852452030095 (Market Val: 193.4292
Guest15 English bids up to: 95.17356307058142
Guest8 joins Dutch (Art)
Guest9 joins Dutch (Art)
Guest15 English bids up to: 108.76978636637877
Guest3 English bids up to: 108.76978636637877
Guest15 English bids up to: 108.76978636637877
Guest17 English bids up to: 108.76978636637877
Guest7 English bids up to: 122.36600966217611
Guest15 English bids up to: 122.36600966217611
Guest17 English bids up to: 122.36600966217611
Guest3 English bids up to: 122.36600966217611
Guest7 English bids up to: 122.36600966217611
Guest17 English bids up to: 122.36600966217611
Guest3 English bids up to: 135.96223295797347
Guest7 English bids up to: 135.96223295797347
Guest15 English bids up to: 135.96223295797347
Guest17 English bids up to: 135.96223295797347
Guest3 English bids up to: 135.96223295797347
Guest7 English bids up to: 135.96223295797347
Guest17 English bids up to: 135.96223295797347
Guest3 English bids up to: 149.55845625377083
Guest7 English bids up to: 149.55845625377083
Guest15 English bids up to: 149.55845625377083
Guest15 English bids up to: 149.55845625377083
Guest15 English bids up to: 163.1546795495682
Guest3 English bids up to: 163.1546795495682
Guest7 English bids up to: 163.1546795495682
Guest15 English bids up to: 163.1546795495682
Guest17 English bids up to: 163.1546795495682
>>> [NEW AUCTION] Auctioneer0 starts English for Art
Guest7 English bids up to: 176.75090284536554
Guest12 joins English (Art)
Guest13 joins English (Art)
Guest17 English bids up to: 176.75090284536554
Auctioneer0: 2 participants joined
Auctioneer1 Cancelled.
Guest12 English bids up to: 77.4403922363654
Guest13 English bids up to: 77.4403922363654
Guest15 English bids up to: 190.3471261411629
Guest17 English bids up to: 190.3471261411629
Guest7 joins Sealed (Toys)
Guest12 English bids up to: 77.4403922363654
Guest13 English bids up to: 77.4403922363654
>>> [NEW AUCTION] Auctioneer4 starts Dutch for Clothes
Guest0 joins Dutch (Clothes)
Guest2 joins Dutch (Clothes)
Guest14 joins Dutch (Clothes)
Guest19 joins Dutch (Clothes)
Auctioneer4: 4 participants joined
```

5.3.2 Auction Site

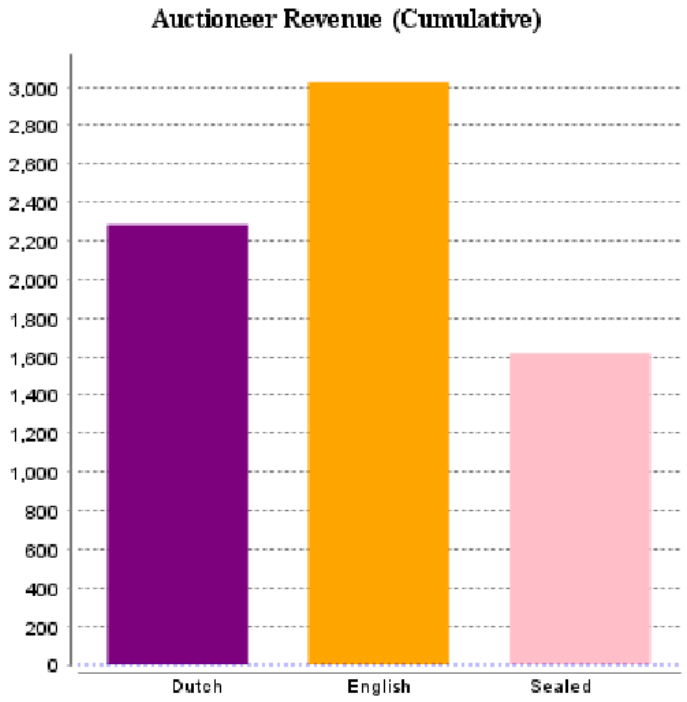


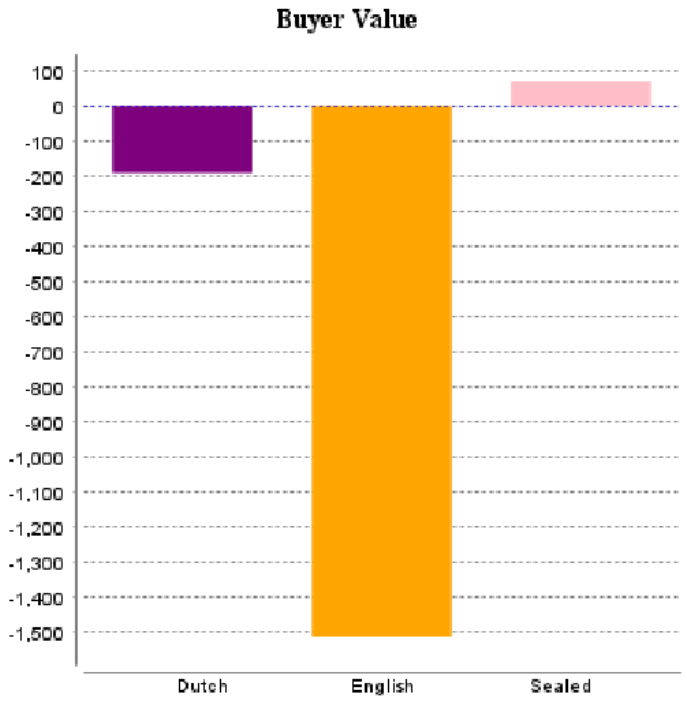
5.3.3 Comparison of Three Auction Returns(auctioneers and buyers)

Dutch Auction:
Fast speed: The price drops rapidly, usually closing in just a few rounds
Auctioneer's income: Medium (buyers wait until a reasonable price to buy)
Buyer surplus: moderate to high (can wait for price drop)
Success rate: High (as long as the price falls within an acceptable range, the transaction will be completed)

English Auction:
Slow speed: requires multiple rounds of bidding, longer in intense competition
Auctioneer's income: high (competition drives up prices)
Buyer surplus: low or negative value (due to bidding wars leading to overspending)
Success rate: Medium (may fail due to high prices)

Sealed Bid Auction:
Fastest speed: Completed in one go
Auctioneer's income: Medium (depending on bidding strategy)
Buyer surplus: significant difference (depending on other people's bids)
Success rate: High (successful as long as someone bids)


Auctioneer Revenue (Cumulative)
3,000
2,800
2,600
2,400
2,200
2,000
1,800
1,600
1,400
1,200
1,000
800
600
400
200
0
Dutch
English
Sealed


Buyer Value
100
0
-100
-200
-300
-400
-500
-600
-700
-800
-900
-1,000
-1,100
-1,200
-1,300
-1,400
-1,500
Dutch
English
Sealed

Section 6: Conclusion

In this assignment, we further explored the GAMA platform by moving from simple reflex agents to complex communicating agents. We learned how to implement the FIPA interaction protocol to enable standardized message passing between Auctioneers and Guests, ensuring that agents negotiate without directly accessing each other's internal variables.

One of the limitations of this work is that the buyers' bidding strategy is relatively static; for example in the dutch auction, they simply buy as soon as the price drops below their expected maximum value. In the real world, buyers often "game" the system by waiting for the price to drop even further, risking that someone else might buy it first. Future work could introduce a risk-taking factor or limited budgets to the guest agents to simulate more realistic behavior.